# Guia de code review

Padrão leve, multi-stack e orientado a risco · Desafio COE Leader · 22/10/2026

## Objetivo

Code review protege comportamento, dados e capacidade de evolução. Compare o diff com a tarefa, a spec e o contrato; verifique riscos do domínio e peça evidência executável. O review complementa testes e CI, não os substitui.

## Bloqueadores antes de aprovar

- Requisito e fronteira: a mudança atende uma tarefa identificada e respeita Pedidos, Catálogo e contratos? Dependências e testes continuam dentro de cada projeto?
- Regra de negócio: o preço vem do Catálogo na criação; os valores usam centavos inteiros; limiar, taxa, arredondamento e limites estão corretos; o histórico preserva snapshots?
- Dados e transações: entradas são não confiáveis; SQL usa parâmetros; cabeçalho e itens confirmam ou revertem juntos; a chamada HTTP ocorre antes da transação?
- Falhas e segurança: há timeout, limites de corpo, status coerentes e logs sem PII, payload ou credenciais? Runtime não recebe permissões de migração?
- Operação e testes: health, cancelamento, paginação e ordem são previsíveis? Testes provam sucesso e falha? Contrato e documentação acompanham a mudança?

## Classifique o comentário

| Nível | Ação esperada |
|---|---|
| Bloqueador (P0/P1) | Corrigir antes do merge: requisito quebrado, risco de segurança/dados, cálculo incorreto, perda de atomicidade ou indisponibilidade. |
| Importante (P2) | Corrigir neste incremento ou registrar responsável e tarefa acordada antes de aprovar. |
| Sugestão (P3) | Opcional e não bloqueia; separe benefício técnico de preferência pessoal. |

Escreva comentários acionáveis: cenário → impacto → mudança esperada. Aponte o menor trecho possível. Pergunte quando faltar contexto. Registre se a pendência foi resolvida, aceita para depois ou rejeitada com justificativa.

# Checklist por projeto

Use os pontos relevantes ao diff e registre riscos residuais no review.

### Pedidos · TypeScript / NestJS / PostgreSQL

- Entrada: DTO fechado, parse estrito, sem conversão implícita nem preço enviado pelo cliente.
- Dinheiro: `bigint` durante cálculo; converter para JSON somente após validar o intervalo seguro.
- Fornecedor: validar conjunto completo, tipos, status, timeout até o fim da resposta e limite de corpo do Catálogo.
- Transação: consultar preço antes de `BEGIN`; usar SQL parametrizado; inserir cabeçalho e itens em lote; liberar conexão em `finally`.
- Histórico: GET consulta snapshot SQL; não chama Catálogo nem recalcula a regra vigente.
- Operação: Problem Details, correlação, logs sanitizados, health checks e migrations revisáveis.

### Catálogo · C# / ASP.NET Core / MongoDB

- Fronteiras: controller fino; domínio sem dependência de HTTP ou do driver Mongo.
- Consulta: propagar `CancellationToken`, limitar tempo e validar documentos lidos como dados não confiáveis.
- Lote: uma busca `$in`, inclui inativos para classificação pelo consumidor e não faz uma busca por produto.
- Eficiência: filtro, projeção, ordem e limites coerentes com índices; medir comandos/`explain` antes de afirmar desempenho.
- Privilégio: runtime tem somente leitura no MongoDB.

### Java · referência para outros serviços

- JDBC: usar `PreparedStatement`, parâmetros de framework e `try-with-resources`; definir limites de transação e timeout. Este exemplo não representa uma terceira aplicação do desafio.

## Evidência mínima e decisão

- Antes de integrar: identificar tarefa/requisitos; executar format, build, análise estática e testes do projeto.
- Teste proporcional: adicionar casos para regras e limites alterados; usar integração real para mudanças de banco/fronteira; rodar smoke Compose nas mudanças integradas.
- Entrega coerente: conferir migrations, OpenAPI, README, configuração e CI no mesmo commit que será integrado.
- Decisão: registrar Aprovar, Solicitar alterações ou Comentar sem bloqueio, com motivo e pendências.

Mentoria acontece em comentários que explicam o risco e em pareamento nas primeiras mudanças de domínio. Acompanhe tendências de tempo de revisão, retrabalho por classe de defeito e falhas escapadas no nível da squad; não ranqueie pessoas.

Se uma verificação externa não foi executada, declare o resultado local e a limitação. Spec, mock e pipeline configurada não comprovam execução real.